UNIVERSITE LIBRE
DE BRUXELLES

SECURE BY DESIGN AND DEFAULT PRINCIPLES

April 2, 2026

SECURE BY DESIGN AND DEFAULT

DEFINITION

- **Security by Design:** embeds protective measures into products during development rather than adding them retrospectively. This includes threat modelling, secure architecture patterns, validated cryptography, and systematic vulnerability management integrated into development processes.

- **Security by Default:** ensures products ship with the most secure configuration reasonably possible.
  - Users should not need technical expertise to achieve baseline security –
  - Protective measures must be active upon installation, with any reduction requiring deliberate user action.

SECURE BY DESIGN AND DEFAULT

SECURITY BY DESIGN PRINCIPLES: ARCHITECTURAL FOUNDATION

Logic and Behaviour:

- 1) how a system is built (Architectural Foundations) and
- 2) how it is managed and maintained (Operational Integrity**)**

Architectural Foundations: establishes how data flows, how components interact, and how the system contains a potential breach before it can spread.

SECURE BY DESIGN AND DEFAULT

SECURITY BY DESIGN PRINCIPLES: ARCHITECTURAL FOUNDATION

Trust boundaries and threat modelling:

- **Trust boundaries** mark where data, identities, or execution contexts move between trust levels (more trusted ↔ less trusted). Examples of boundaries include:
  - Device ↔ cloud service
  - User ↔ admin interface
  - One microservice ↔ another
- Boundaries help define:
  - Which components must authenticate each other
  - Where input should be treated as untrusted
  - Where extra controls (validation, rate limiting, encryption, isolation) are needed

- **Threat modelling** adds analyze boundary risks by:
  - Identifying key assets (credentials, firmware images, customer data)
  - Considering likely threat actors
  - Evaluating attacker capabilities and assumptions

SECURE BY DESIGN AND DEFAULT

SECURITY BY DESIGN PRINCIPLES: ARCHITECTURAL FOUNDATION

Strong identity and authentication architecture:

- ✓ Clear and consistent approach how identities are created, verified, and managed for users,
- ✓ devices, services, and administrators +
- ✓ Ensuring that authentication is resistant to common attacks such as credential stuffing, replay, and session hijacking.

SECURE BY DESIGN AND DEFAULT

SECURITY BY DESIGN PRINCIPLES: ARCHITECTURAL FOUNDATION

Attack Surface Minimization: Unnecessary features, services, and interfaces increase potential attack vectors.

Reducing system complexity and disabling unused components lowers vulnerability risks.

- Removing **default accounts**
- Uninstalling **unused packages**
- Closing **nonessential ports**
- Removing **unused code**
- Limiting **management interfaces to trusted networks only**
- Use **standardized secure baselines and hardened configuration templates** to keep systems consistently minimal **as they scale**.
- **Perform ongoing vulnerability scanning and maintain an accurate asset inventory** (because it is impossible to minimize what you do not know exists.

SECURE BY DESIGN AND DEFAULT

SECURITY BY DESIGN PRINCIPLES: ARCHITECTURAL FOUNDATION

Attack Surface Minimization: Unnecessary features, services, and interfaces increase potential attack vectors.

Reducing system complexity and disabling unused components lowers vulnerability risks.

- Removing **default accounts**
- Uninstalling **unused packages**
- Closing **nonessential ports**
- Removing **unused code**
- Limiting **management interfaces to trusted networks only**
- Use **standardized secure baselines and hardened configuration templates** to keep systems consistently minimal **as they scale**.
- **Perform ongoing vulnerability scanning and maintain an accurate asset inventory** (because it is impossible to minimize what you do not know exists.

SECURE BY DESIGN AND DEFAULT

SECURITY BY DESIGN PRINCIPLES: ARCHITECTURAL FOUNDATION

Defence in depth uses layered security controls so one failure doesn't cause full compromise.

Layers include:

- **Preventive controls: MFA, network segmentation**
- **Detective controls: monitoring, logging, anomaly detection**
- **Corrective controls: automated isolation, backup/restore**

Assumes some controls may be bypassed, so systems should degrade gracefully while protecting critical assets.

Reduces dependence on any single "silver bullet" by increasing attacker cost and effort.

Most effective when controls are diverse, avoiding reliance on the same technology or trust assumptions.

SECURE BY DESIGN AND DEFAULT

SECURITY BY DESIGN PRINCIPLES: ARCHITECTURAL FOUNDATION

Open Design (avoid obscurity): Systems should not rely on secrecy of design or hidden behavior for protection.

Security controls must remain effective when adversaries understand how the system operates.

The principle promotes the use of:

- Well-studied algorithms and protocols.
- Clear and transparent documentation.
- Designs that can withstand scrutiny through review and testing.

"Open design" means security should depend on:

- Protected keys.
- Strong authentication.
- Robust implementation.
- Not on keeping mechanisms secret.

Transparent designs improve maintainability by making systems easier to audit, validate, and enhance over time.

SECURE BY DESIGN AND DEFAULT

SECURITY BY DESIGN PRINCIPLES: OPERATIONAL INTEGRITY

Operational Integrity = human and procedural factors that maintain system security throughout its lifecycle.

- It **covers all stages**—from the first line of code to the system's retirement.
- It ensures security **is a continuous, ongoing practice, not a one-time effort**.
- Both **developers and users share responsibility for maintaining this continuous security mindset**

Lifecycle management:

User-Centric Design:

Supply Chain Control

Secure Coding Practices

Configuration and change management:

Logging, Monitoring and Alerting

SECURE BY DESIGN AND DEFAULT

SECURITY BY DESIGN PRINCIPLES: OPERATIONAL INTEGRITY

Lifecycle management:

- **Security responsibilities extend beyond initial development**.
  Components must be maintained, updated and eventually retired in a controlled manner, taking the expected duration of use into account.

User-Centric Design:

- **User-centric design** = security mechanisms are usable and understandable for everyday users
- **Example**: If encryption requires complex manual configuration, users may skip it or configure it incorrectly. A simple, guided setup that enables encryption automatically reduces misconfiguration risks. Making security features easy to use increases the likelihood that users will apply them correctly but Poor usability can lead to insecure workarounds or misconfigurations.

SECURE BY DESIGN AND DEFAULT

SECURITY BY DESIGN PRINCIPLES: OPERATIONAL INTEGRITY

Secure Coding Practices:

- **Prevent:** Developers must **follow secure coding standards** to prevent common vulnerabilities.
- **Detect**: Early detection and mitigation of insecure code prevent weaknesses from being built into the system.
- **Detect:** Use Static Application Security Testing (SAST) during coding to identify vulnerabilities early.
- **Detect :** Use Software Composition Analysis (SCA) to detect vulnerable third-party libraries.
- **Detect :** Apply Dynamic Application Security Testing (DAST) before deployment to uncover runtime issues.

SAST : **Static Application Security Testing**= scans source code, bytecode, or binaries to find vulnerabilities **without running the application. SAST = having a code reviewer before the app ever runs. False positives exist** because SAST does not see the application running in real conditions

DAST: **Dynamic Application Security Testing** = tests a running application from the outside, like an attacker, to find security weaknesses without access to the source code. **it focuses on runtime behavior . It sends automated requests or attacks to the live application and observes the responses + unexpected behavior.**

SCA : Software Composition Analysis = identifies the open-source and third-party components , then checking them for known vulnerabilities, license issues, and supply-chain risk. It builds an inventory of dependencies and components in the software + Matches those components against vulnerability databases such as NVD and CVE sources.

Logging, Monitoring and Alerting: Security depends on visibility.

- Systems generate security-relevant logs.
- Logs should **be retained for a defined period**.
- Logs should **be protected from tampering to support investigation and compliance.**
- **Monitoring does more than collect data**: detect suspicious behavior.
- Examples of suspicious behavior:
  - Repeated failed logins,
  - Privilege escalation,
  - Unexpected configuration changes, and
  - Unusual outbound connections.

SECURE BY DESIGN AND DEFAULT

SECURITY BY DESIGN PRINCIPLES: OPERATIONAL INTEGRITY

Configuration and change management: Secure operation requires configurations to be controlled, consistent, and auditable

- **Baseline hardening standards** should be defined (for example, secure defaults, disabled unused services, and enforced encryption settings)
- **Baseline applied through repeatable mechanisms such as templates and infrastructure-as-code** (means servers, networks, and security settings are described in code, so they can be deployed automatically and consistently)

Incident response and recovery

- **Developers ready to respond to security incidents** affecting their products in the field via internal roles defined + **Decision-making authority** should be assigned for handling **security incidents** + **Clear Escalation paths** for security events
- **Playbooks for containment** and **customer communication.**
- Incidents = **vulnerabilities, compromised code, malicious updates, and misuse of product functionality.**

Supply Chain Control

- Focus on high-impact areas: **code repositories, build systems, signing keys, and update distribution channels**.
- Minimum controls for source code and CI/CD access:
  - ✓ *Limit **access to named individuals**.*
  - ✓ *Require **multi-factor authentication**.*
  - ✓ *Implement **lightweight peer approval for changes to security-critical areas**.*
  - ✓ *Generate and maintain SBOMs (Software Bill of Materials) for dependency transparency, vulnerability management, and product lifecycle security.*

SBOM : Software Bill of Materials. It's a formal, machine-readable inventory listing all components, dependencies, libraries, and modules that make up a software application—including open-source and third-party elements

CI/CD : Continuous Integration/Continuous Delivery (or Deployment) = Automated practices streamlining software development by frequently integrating code changes, testing them, and deploying updates reliably.

Continuous Integration (CI): Developers merge code changes into a shared repository multiple times a day; automated builds and tests run immediately to catch integration issues early.

Continuous Delivery (CD): Automates code delivery to a staging or production-ready environment after passing tests, but requires human approval for final release.

Definition

- Security by default complements security by design by focusing on the product's security configuration presented to the use
- Systems can become vulnerable if released with insecure or overly permissive defaults settings.

- **Goal of security by default** : minimise the attack surface after deployment by ensuring that the most secure configuration is applied automatically
- **Secure defaults reduce reliance on user expertise** and limit the potential for misconfigurations, which are a common source of security incidents.

SECURITY BY DEFAULT PRINCIPLES: DEFAULT HARDENING

Definition

- The default hardening category focuses on the factory-shipped state of the software.
- Initial configuration is as restrictive as possible

How to ?

- **Minimisation of Default Services**: Any feature or service not essential for the core functionality of the product should be disabled by default.
  E.g. If a web server includes an optional file-sharing module that most users won't need, that module should be "off" until the user explicitly opts in (=reduction of immediate attack surface).

- **Secure Communication by Default:** All external communications should be encrypted and authenticated from the first connection. Rather than allowing an unencrypted HTTP or Telnet connection "for convenience," the system should strictly enforce protocols like TLS 1.3 or SSH

How to ? (continued)

- **Unique device identity and secrets by default:** The product should ship with unique, per device credentials and cryptographic identity (keys/certificates), not shared defaults.
  Any secrets used for authentication, update verification, or encrypted communications must be generated uniquely and protected against extraction.

SECURITY BY DEFAULT PRINCIPLES: GUIDED PROTECTION

Definition

- Guided Protection addresses the interaction between the user and the system's security features

How to ?

- **Mandatory Security Onboarding***: Critical security features should not be "hidden"* in a settings menu; they should be part of the initial setup wizard.
  For instance, requiring a user to configure Multi-Factor Authentication (MFA) or an encryption key during the first-run experience ensures that the system enters a secure state before it is exposed to the internet.
- **Automated Maintenance and Updates:** A secure default posture must be sustainable.
  By enabling automatic security updates by default, the manufacturer ensures that the product remains protected against newly discovered vulnerabilities without requiring to have a dedicated IT team to manually manage patching cycles.

SECURITY BY DEFAULT PRINCIPLES: GUIDED PROTECTION

Definition

- Guided Protection addresses the interaction between the user and the system's security features

How to ?

- **Transparent Security Posture:** The system should clearly communicate its security status to the user. If a user chooses to disable a security feature or if a specific configuration increases risk, the system must provide a clear, understandable warning and offer a "one-click" path to return to the secure baseline.

- **Secure Recovery and Ownership Lifecycle:** The product should provide guided, low-friction recovery and transfer processes (credential reset, account recovery, secure factory reset, and ownership transfer) that are simple for users to follow but resistant to account takeover and social engineering.